Phase 1 - Grade 3 - Pandas DataFrames

Key Learning Points

A NumPy array has no concept of named columns or per-column types. Real-world data looks like an Excel sheet or SQL table: named columns, each potentially a different type, bundled into one object. Pandas' **DataFrame** gives Python that Excel-sheet structure, built on top of NumPy under the hood.

Analogy that made it click: if a NumPy array is like a Java int[][], a DataFrame is like a List<Map<String,Object>> or a SQL ResultSet -- rows of structured, named, mixed-type fields.

DataFrame
: 2-D, labeled tabular structure. Columns have names and can each hold a different type; rows are records. Built internally on NumPy arrays.

Series
: A single column (or row) of a DataFrame -- effectively a 1-D NumPy array with labels attached. All values share the same type.

.iloc[n]
: Selects by INTEGER POSITION -- the physical location, counting from 0, regardless of the row's index label.

.loc[label]
: Selects by INDEX LABEL -- whatever that label literally is. Labels and physical positions usually match on a fresh DataFrame, but diverge after filtering, sorting, or re-indexing.

.idxmax() / .idxmin()
: Returns the INDEX LABEL of the row holding the max/min value in a column -- the idiomatic way to jump to the extreme-value row directly.

.loc vs .iloc -- the mechanism

.loc is label-based: looks up a row by its literal index label. .iloc is integer-position-based: always means physical position N, counting from 0, no matter what the labels say. On a fresh DataFrame these coincide, but after df2 = df[df['dept']=='Eng'] filters out some rows, labels can become non-sequential (e.g. only rows 0 and 2 remain). Then df2.loc[1] errors (no label '1' left), while df2.iloc[1] still works (means 'the 2nd row physically present now').

Clarified Doubts

My doubt: For filtering by department, do you want only the employee names, or the entire matching row?
: Clarification: The ENTIRE row (all columns) for every matching record -- same as how boolean masking already worked: df[df['stock'] > 0] returns full rows, not just one column.

My doubt: Does '.iloc[2]' (asked as 'row at position 2') mean the 2nd row (counting) or index position 2?
: Clarification: It means INDEX 2 -- zero-indexed, same rule as NumPy. '.iloc[2]' is the THIRD physical row, not the second. Going forward, say 'index N' explicitly to avoid this ambiguity.

Examples

```
import pandas as pd

data = {'product': ['Pen','Notebook','Eraser','Marker'],
        'price':   [10, 40, 5, 25],
        'stock':   [100, 30, 200, 0]}
df = pd.DataFrame(data)

df['price']              # column -> Series
df.iloc[2]                # row at INTEGER POSITION 2 -> Eraser
df[df['stock'] > 0]      # boolean masking, full rows

df['tot'] = df['price'] * df['stock']          # computed column
df[df['tot'] == df['tot'].max()]                # filter for the max (2 steps)
df.loc[df['tot'].idxmax()]                      # cleaner: idxmax jumps straight there

df['price'].mean()
```

Quiz Q&A;

Q1. Get only employees in the 'Eng' department (full rows).

A: df[df['dept'] == 'Eng']

Q2. Add a new column bonus = 10% of salary.

A: df['bonus'] = df['salary'] * 0.1

Q3. Get the full row with the lowest salary, using .idxmin().

A: df.loc[df['salary'].idxmin()]

Q4. What is the difference between a pandas Series and a DataFrame?

A: Series is 1-D, a single column/row of same-type data. DataFrame is 2-D, multiple columns, each potentially a different type.

Q5. True or False: df.iloc[0] and df.loc[0] will always return the exact same row, no matter what.

A: False. .loc looks up by label; .iloc looks up by integer position. They only coincide when the DataFrame's labels haven't been touched (e.g. before filtering/sorting) -- afterward they can diverge.